

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE:CSA17

COURSE OUTCOME COVERED

C01: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Geetika T (192511469) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one.

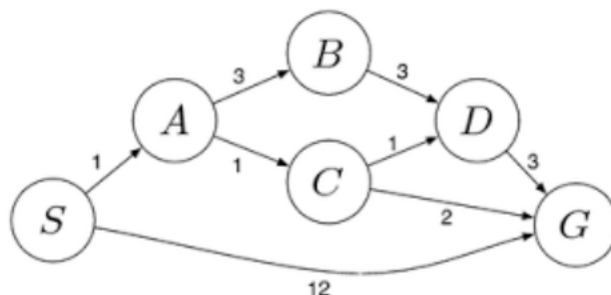
Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug?

Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

- What are the percept's for this agent?
- Characterize the operating environment.
- What are the actions the agent can take?
- How can one evaluate the performance of the agent?
- What sort of agent architecture do you think is most suitable for this agent?
Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution because two queens share the same diagonal as the queen in the last column.



4. A customer wants to use OLA Cab booking mobile app to book a cab of type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.
- The delivery network is represented as a weighted graph:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Question 1 – Water Jug Problem

A. Problem understanding

There are two unmarked jugs: Jug X holds 4 gallons and Jug Y holds 3 gallons. A pump can fill either jug; water can be discarded; and water can be poured between the jugs until the source is empty or the receiving jug is full. The goal is to leave exactly 2 gallons in Jug X.

State: (x, y) , where x is the amount in the 4-gallon jug and y is the amount in the 3-gallon jug; $0 \leq x \leq 4$ and $0 \leq y \leq 3$.

Initial state: $(0, 0)$.

Goal test: $x = 2$; the amount in Jug Y does not matter.

Action cost: one unit per fill, empty or pour action, so the objective is to use as few actions as possible.

There are $5 \times 4 = 20$ possible states. Starting from $(0, 0)$, the search reaches 14 distinct states before it finds a goal state.

B. Method selection

I use breadth-first search because every action has the same cost. BFS explores all states at depth 0, then depth 1, and so on; the first goal it finds is therefore a shortest-action solution. The state space is small, and keeping a visited set prevents loops. Depth-first search could find a solution, but it does not guarantee the fewest actions.

C. Solution process

From state (x, y) , the available operations are:

- Fill Jug X: $(4, y)$; fill Jug Y: $(x, 3)$.
- Empty Jug X: $(0, y)$; empty Jug Y: $(x, 0)$.
- Pour X into Y by $t = \min(x, 3 - y)$, giving $(x - t, y + t)$.
- Pour Y into X by $t = \min(y, 4 - x)$, giving $(x + t, y - t)$.

The BFS layers around the shortest path are: depth 0, $(0,0)$; depth 1, $(4,0)$ and $(0,3)$; depth 2, $(1,3)$, $(3,0)$, and $(4,3)$; depth 3, $(1,0)$ and $(3,3)$; depth 4, $(0,1)$ and $(4,2)$; depth 5, $(4,1)$ and $(0,2)$. At depth 6, both $(2,3)$ and $(2,0)$ satisfy the goal test.

One shortest path is:

0. Start at $(0, 0)$ — both jugs are empty.

1. Fill Jug X — $(0,0) \rightarrow (4,0)$.

2. Pour X into Y — $t = \min(4, 3 - 0) = 3$, so the state becomes $(4 - 3, 0 + 3) = (1,3)$.

3. Empty Jug Y — $(1,3) \rightarrow (1,0)$; 3 gallons are discarded.

4. Pour X into Y — $t = \min(1, 3 - 0) = 1$, giving $(0,1)$.

5. Fill Jug X — $(0,1) \rightarrow (4,1)$.

6. Pour X into Y — $t = \min(4, 3 - 1) = 2$, giving $(4 - 2, 1 + 2) = (2,3)$. The goal is reached.

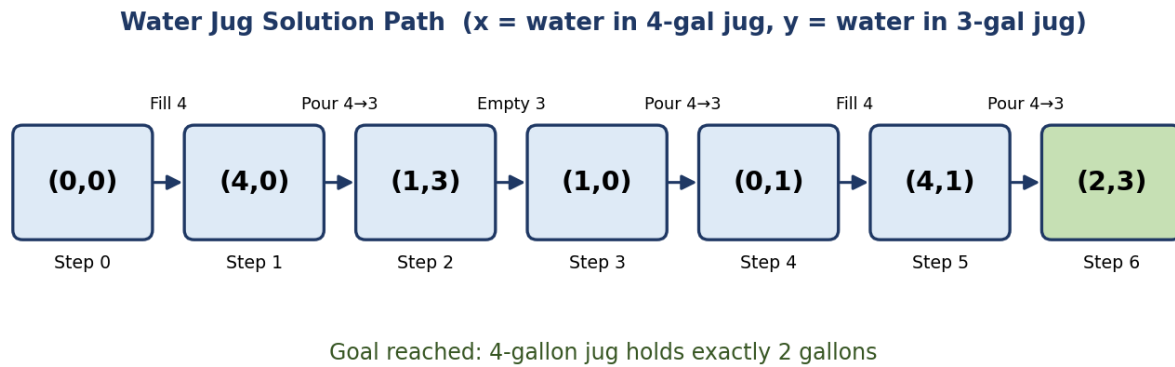


Figure 1.1. Shortest water-jug solution path from (0,0) to (2,3).

D. Accuracy of calculation

The pump adds 4 gallons at Step 1 and another 4 gallons at Step 5, so 8 gallons have been drawn in total. The only discarded water is the 3 gallons emptied from Jug Y at Step 3. At the end, the jugs contain $2 + 3 = 5$ gallons. The balance checks: $8 \text{ pumped} - 3 \text{ discarded} = 5 \text{ in the jugs}$.

The IDLE run below independently returns a six-action path and reports 14 generated states. Its final state is (2,3), so Jug X holds the required 2 gallons.

BFS check: program and run in Python IDLE

The editor shows `jug.py`; the Shell screenshot below is the result from running that file in IDLE (Python 3.12.3).

```

jug.py /home/ubuntu/words/newsworld/jug.py (3.12.1)
File Edit Format Run Options Window Help

from collections import deque

CAP_X, CAP_Y, GOAL = 4, 3, 2

def successors(state):
    x, y = state
    moves = []
    [(CAP_X, y), "Fill 4-gallon jug"),
    [(x, CAP_Y), "Fill 3-gallon jug"),
    [(0, y), "Empty 4-gallon jug"),
    [(x, 0), "Empty 3-gallon jug"),
    ]
    amount = min(x, CAP_Y - y)
    moves.append([(x - amount, y + amount), "Pour 4 -> 3"])
    amount = min(y, CAP_X - x)
    moves.append([(x + amount, y - amount), "Pour 3 -> 4"])
    return moves

def bfs(start=(0, 0)):
    frontier = deque([start])
    parent = {start: (None, "Start()")}
    while frontier:
        state = frontier.popleft()
        if state[0] == GOAL:
            path = []
            while state is not None:
                previous, action = parent[state]
                path.append((state, action))
                state = previous
            return path[::-1], len(parent)
        for next_state, action in successors(state):
            if next_state not in parent:
                parent[next_state] = (state, action)
                frontier.append(next_state)
    return None, len(parent)

path, generated = bfs()
for step, (state, action) in enumerate(path):
    print(f"Step {step}: {state} {action}")
print(f"Actions:", len(path) - 1)
print(f"States generated:", generated)

```

```

Step 0: (0, 0) Start
Step 1: (4, 0) Fill 4-gallon jug
Step 2: (1, 3) Pour 4 -> 3
Step 3: (1, 0) Empty 3-gallon jug
Step 4: (0, 1) Pour 4 -> 3
Step 5: (4, 1) Fill 4-gallon jug
Step 6: (2, 3) Pour 4 -> 3
Actions: 6
States generated: 14

```

Python IDLE Shell output

E. Interpretation of results

The 4-gallon jug contains exactly 2 gallons after six actions: fill X, pour $X \rightarrow Y$, empty Y, pour $X \rightarrow Y$, fill X, then pour $X \rightarrow Y$. BFS makes the six-action result optimal because it reaches the goal at the first depth where any goal is available. The puzzle is measurable because $\text{gcd}(4, 3) = 1$; a reverse-direction solution starting with the 3-gallon jug also takes six actions.

Question 2 — Mars Rover as an Intelligent Agent

A. Problem understanding

A Mars rover must move safely over unfamiliar terrain, examine rocks and soil, collect samples, and send useful data to Earth. Since a radio signal takes several minutes to travel between Earth and Mars, the rover cannot rely on continuous joystick control. It must use its sensors and onboard software to make many decisions locally.

B. Method selection

PEAS is a useful starting point because it makes the task specification explicit: Performance measure, Environment, Actuators and Sensors. I then describe the environment properties and select a layered agent architecture. A single reflex rule would be too limited for a mission involving hidden terrain, delayed communication and competing goals.

C. Solution process

2(a) PEAS description

Performance measure: maximize scientific value, useful samples, area explored, uptime and data quality; minimize energy use, wasted time, collision, wheel-stuck events and tip-overs.

Environment: Martian terrain—rocks, soil, slopes, craters, dust, sunlight and weather—with incomplete maps and a communication delay of roughly 4–24 minutes one way.

Actuators: drive and steering motors, robotic arm and turret, drill/corer, sample-handling mechanism, camera mast, communication antenna and power/thermal controls.

Sensors: stereo and hazard cameras, spectrometers such as APXS/ChemCam, inertial and tilt sensors, wheel encoders and motor-current sensors, weather sensors, battery readings, radio receiver and internal health sensors.

A simplified percept at time t can be written as [image, spectrum, tilt, wheel odometry, temperature, battery, command].

2(b) Environment properties

- Partially observable: cameras see only part of the surroundings, and the subsurface is hidden.
- Single-agent for this model: orbiters and other rovers may cooperate, but they are not adversaries in this task.
- Stochastic: wheel slip, loose sand and rock stability make outcomes uncertain.
- Sequential: a drive decision changes position, energy and later sample opportunities.
- Dynamic: dust, lighting, temperature and sunlight can change while the rover plans.
- Continuous and partly unknown: location, speed and time are continuous, while the terrain map is incomplete.

2(c) Actions

The rover can drive, turn, stop, climb or back away; aim its instruments; drill and collect a sample; transfer or store samples; point and zoom its cameras; transmit data or receive commands; and enter a low-power or safe mode.

2(d) Performance calculation

One illustrative score is: $\text{Performance} = w_1(\text{science value}) + w_2(\text{samples analysed}) + w_3(\text{distance driven safely}) - w_4(\text{energy used}) - w_5(\text{time wasted}) - w_6(\text{hazards})$. The weights express mission priorities and must be chosen with consistent units.

For one sol, take weights 5, 3, 0.02, 0.01, 0.1 and 20. Suppose the rover records science value 4, analyses 2 samples, drives 50 m safely, uses 300 Wh, wastes 10 minutes and has no hazards:

$\text{Score} = 5(4) + 3(2) + 0.02(50) - 0.01(300) - 0.1(10) - 20(0) = 20 + 6 + 1 - 3 - 1 - 0 = 23$ points.

If one collision occurs, the score falls by 20 points to 3, showing that the chosen weights strongly prioritize safety.

2(e) Agent architecture

The most suitable design is a utility-based, goal-directed agent with an internal model, a fast safety layer and a learning component. The model tracks location, slopes and battery state; the planner turns a goal such as “analyse rock R” into a sequence of moves; and a utility function weighs science against energy and risk. Reflex rules can immediately stop the rover when tilt or wheel-slip readings become unsafe. Learning can improve terrain classification from experience.

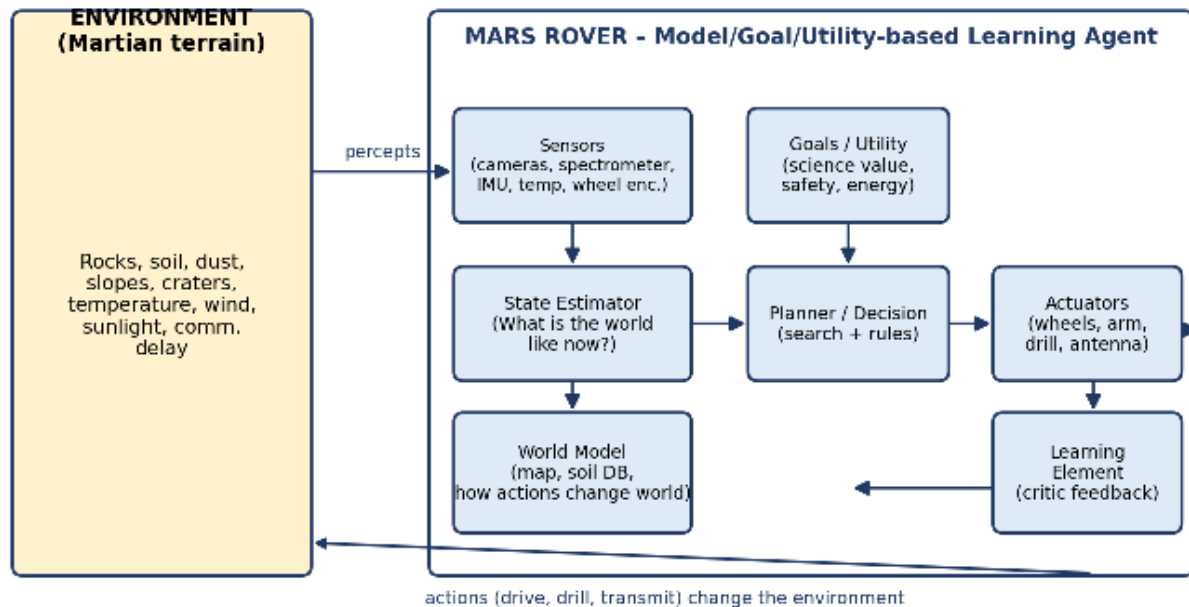


Figure 2.1. A layered rover agent: percepts feed the model and planner; actions affect the environment.

D. Accuracy of calculation

The worked score is consistent: the positive terms total $20 + 6 + 1 = 27$, and the penalties total $3 + 1 + 0 = 4$. Therefore $27 - 4 = 23$ points. A single hazard penalty of 20 would reduce that to 3 points.

E. Interpretation of results

PEAS clarifies what the rover must sense, what it can do and what counts as mission success. Because the environment is only partly known and actions have delayed consequences, the rover needs planning and a world model not just isolated if/then rules. The numerical score is illustrative; its practical value depends on units and agreed mission weights.

Question 3 — Eight Queens: Formulation and Search

A. Problem understanding

Place eight queens on an 8×8 chessboard so that no pair shares a row, a column or a diagonal. In the given example, the arrangement is not valid because the queens in the first and last columns share a diagonal.

Goal: place all eight queens with zero attacking pairs ($h = 0$).

Attack test: queens at (r_1, c_1) and (r_2, c_2) attack if $r_1 = r_2$, $c_1 = c_2$ or $|r_1 - r_2| = |c_1 - c_2|$.

A naive placement of eight queens on any of the 64 squares has $C(64, 8) = 64! / (8! \cdot 56!) = 4,426,165,368$ arrangements.

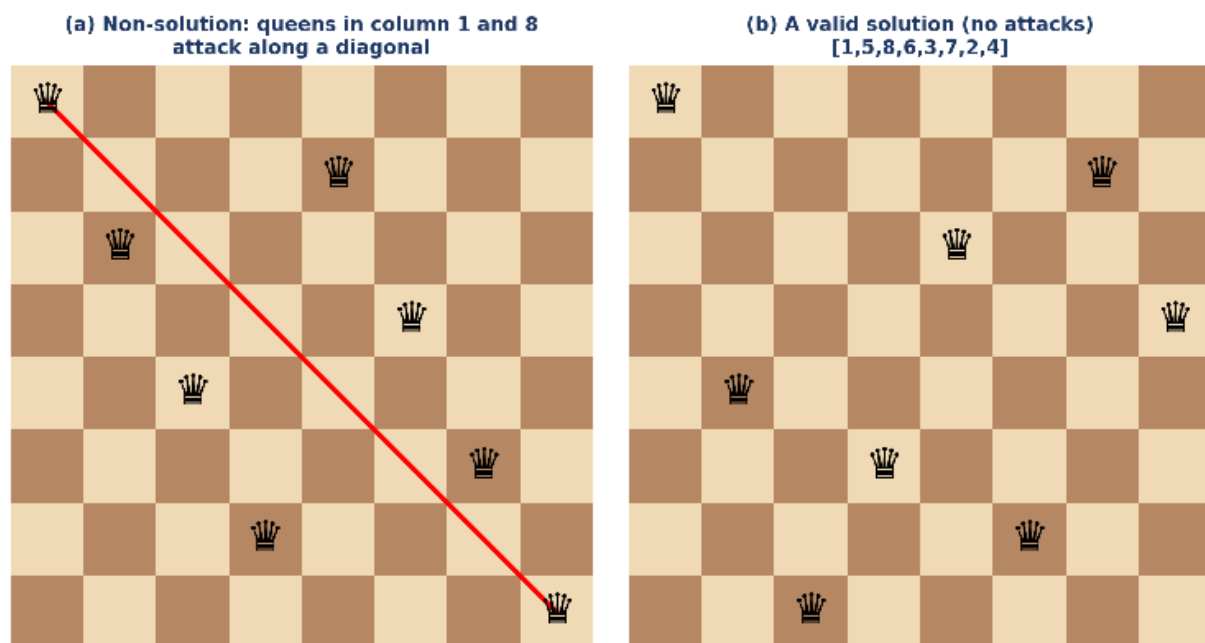


Figure 3.1. The first board has a diagonal attack; the second is a valid solution.

B. Method selection

I use the incremental formulation: start with an empty board and place one queen in each new row, only in a column that is not attacked. Depth-first search with backtracking fits this formulation because the depth is at most eight. A complete-state formulation with hill climbing can be useful for larger local-search tasks, but it may get stuck at a local minimum; DFS with backtracking is complete for this finite search.

C. Solution process

Initial state: empty board.

State: a safe partial placement of k queens in the first k rows.

Action: place a queen in a safe column of the next row.

Transition: add that queen to the board.

Goal test: eight queens have been placed and no pair attacks.

Path cost: not important here; only the final arrangement matters.

Why the incremental formulation is smaller

The same problem can be counted at several levels: $8^8 = 16,777,216$ arrangements if each column independently chooses a row; $8! = 40,320$ if there is one queen in every row and column; and $C(64,8) = 4,426,165,368$ when any eight squares are allowed. With backtracking, only safe partial placements are extended. The counts by depth are 1, 8, 42, 140, 344, 568, 550, 312 and 92. Adding them gives $1 + 8 + 42 + 140 + 344 + 568 + 550 + 312 + 92 = 2,057$ safe states.

A short backtracking trace

- Row 1: all columns are safe, so try column 1.
- Row 2: columns 1 and 2 are attacked; try column 3.
- Row 3: safe choices are 5, 7 and 8; try 5.
- Rows 4 and 5: try columns 2 and 4, respectively.
- Row 6: no column is safe, so backtrack. Trying the next choice in row 5 also fails; DFS then backs up farther and continues.

With columns tried from left to right, the first complete solution is [1, 5, 8, 6, 3, 7, 2, 4], meaning row 1 uses column 1, row 2 uses column 5, and so on.

For this placement, the columns are all different. The row-plus-column values are {2, 7, 11, 10, 8, 13, 9, 12}, and the row-minus-column values are {0, -3, -5, -2, 2, -1, 5, 4}; neither set has a repeated value. Therefore there is no diagonal attack.

D. Accuracy of calculation

For the first pictured arrangement, the row positions by column are [1, 3, 5, 7, 2, 4, 6, 8]. The queens in columns 1 and 8 lie on the same diagonal, so $h = 1$ and the board fails the goal test. The valid solution has $h = 0$. The executed DFS program confirms 92 total solutions and reports 113 placements before its first solution for this column order.

The search-space reduction is $4,426,165,368 \div 2,057 \approx 2,151,658$. This is a little over 2.1 million times fewer states than checking all eight-square arrangements.

DFS/backtracking check: program and run in Python IDLE

The editor shows queens.py; the Shell screenshot below is the result from running that file in IDLE (Python 3.12.3).

```
queens.py - /home/submit/work/answer notebook/queens.py (3.12.3)
File Edit Format Run Options Window Help

N = 8

def attacks(placed, row, col):
    for old_row, old_col in enumerate(placed):
        if old_col == col or abs(old_col - col) == row - old_row:
            return True
    return False

def solve(placed=[]):
    row = len(placed)
    if row == N:
        yield placed
        return
    for col in range(N):
        if not attacks(placed, row, col):
            yield from solve(placed + [col,])

def first_solution_placements():
    count = 0
    def visit(placed):
        nonlocal count
        row = len(placed)
        if row == N:
            return True
        for col in range(N):
            if not attacks(placed, row, col):
                count += 1
                if visit(placed + [col,]):
                    return True
        return False
    visit([])
    return count

def conflicts(positions):
    return sum(
        1 for r1 in range(N) for r2 in range(r1 + 1, N)
        if positions[r1] == positions[r2]
        or abs(positions[r1] - positions[r2]) == r2 - r1
    )

solutions = list(solve())
figure_a = (8, 2, 4, 6, 1, 3, 5, 7)
print("Total solutions:", len(solutions))
print("First solution (row -> column):", [c + 1 for c in solutions[0]])
print("Placements to first solution:", first_solution_placements())
print("Conflicts in figure (a):", conflicts(figure_a))
print("Conflicts in a solution:", conflicts(solutions[0]))
```

Python IDLE editor

```
>>> Total solutions: 92
First solution (row -> column): [1, 5, 8, 6, 3, 7, 2, 4]
Placements to first solution: 113
Conflicts in figure (a): 1
Conflicts in a solution: 0
```

Python IDLE Shell output

E. Interpretation of results

Backtracking avoids building partial boards that already contain an attack. It therefore reduces the explored state count substantially, while still finding all 92 solutions. The same “extend only valid partial choices” idea is useful in timetabling, layout and resource-allocation problems.

Question 4 — OLA Cab Booking: Problem-Solving Agent

A. Problem understanding

A rider enters a pickup and destination, then needs a suitable ride based on passenger count, price, waiting time and comfort. The booking system receives location, traffic, cab availability, fares and the rider’s preferences. Its actions include presenting options, confirming a cab, assigning a driver and arranging the trip.

Goal: get the rider to the destination in an available cab that meets the rider’s constraints and offers an acceptable balance of fare, arrival time and comfort.

Environment: dynamic and partly observable, with traffic, changing cab availability, multiple drivers

and riders, and outcomes that unfold over the trip.

B. Method selection

A plain goal-based agent can tell whether the rider reaches the destination, but it does not distinguish between a cheaper cab and a more comfortable one. A utility-based problem-solving agent is a better fit: it first keeps only available cabs with enough seats, then ranks the suitable options using the rider's priorities.

C. Solution process

The agent follows the usual problem-solving cycle: update the current state from the rider's request and live information; formulate the goal of reaching the destination; build the feasible cab choices; estimate fare and arrival time; score each cab; then confirm the selected option and continue with driver assignment, ride and payment. If no available cab has enough seats, it reports that no suitable cab is available.

Utility model and worked comparison

For the illustrative example, use $U(c) = 0.5(1 - \text{fare}/600) + 0.2(1 - \text{ETA}/20) + 0.3(\text{comfort}/10)$. Here fare is in rupees, ETA is in minutes, and comfort is rated from 1 to 10. The weights sum to 1, and larger utility is preferred.

- Micro: $0.350 + 0.150 + 0.150 = 0.650$.
- Mini: $0.317 + 0.160 + 0.180 = 0.657$.
- Sedan: $0.233 + 0.130 + 0.240 = 0.603$.
- Prime: $0.150 + 0.110 + 0.270 = 0.530$.
- Share: $0.408 + 0.080 + 0.090 = 0.578$.

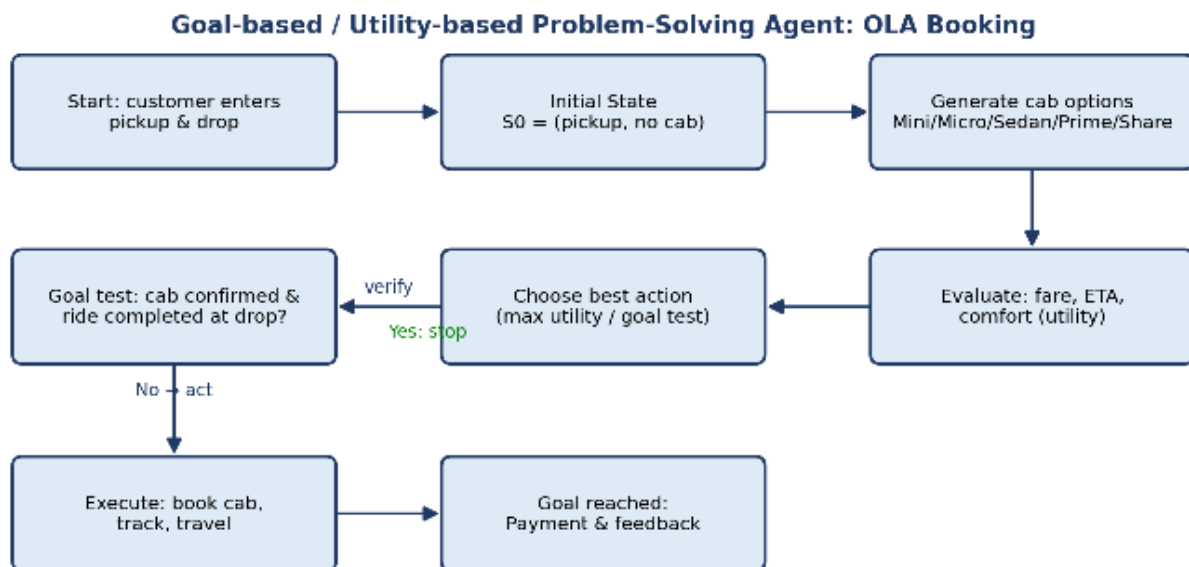


Figure 4.1. The booking agent filters options, evaluates utility and carries out the selected plan.

D. Accuracy of calculation

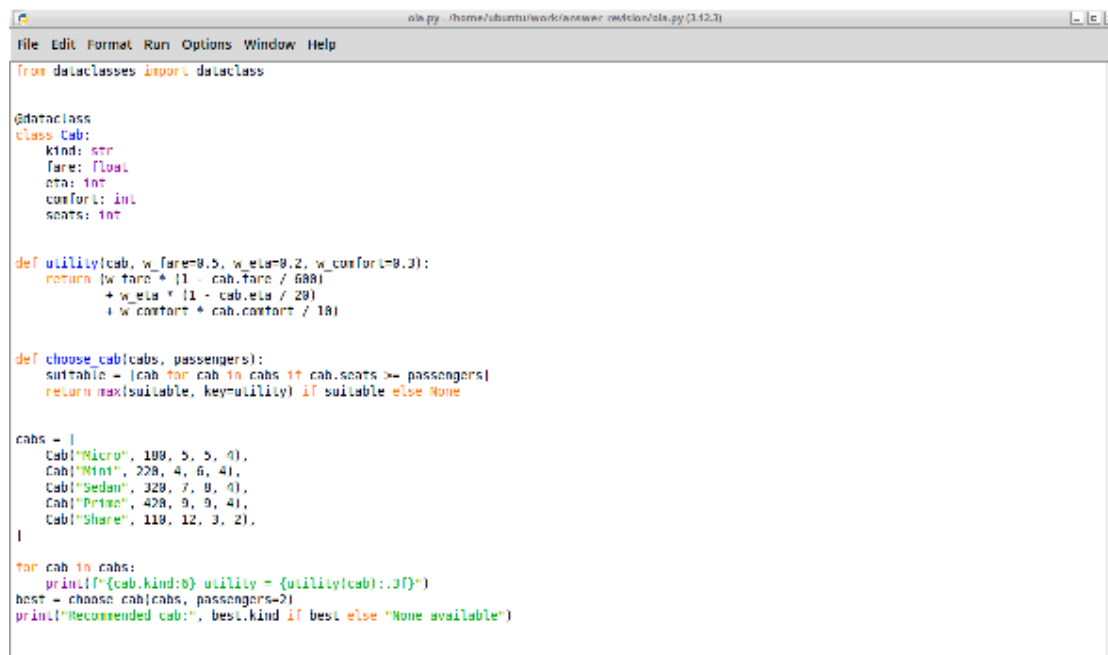
For Mini (fare ₹220, ETA 4 minutes, comfort 6), the fare contribution is $0.5 \times (1 - 220/600) = 0.3167$; the ETA contribution is $0.2 \times (1 - 4/20) = 0.1600$; and the comfort contribution is $0.3 \times (6/10) = 0.1800$. Thus $U(\text{Mini}) = 0.6567 \approx 0.657$. This is higher than Micro's 0.650, so Mini is

selected in this example. The values are illustrative, not live OLA prices.

The IDLE program prints the utility for each cab and chooses the highest-scoring available option with enough seats.

Utility calculation check: program and run in Python IDLE

The editor shows ola.py; the Shell screenshot below is the result from running that file in IDLE (Python 3.12.3).



```
ola.py - /home/ubuntu/work/olawar/medclan/ola.py (3.12.3)
File Edit Format Run Options Window Help
from dataclasses import dataclass

@dataclass
class Cab:
    kind: str
    fare: float
    eta: int
    comfort: int
    seats: int

def utility(cab, w_fare=0.5, w_eta=0.2, w_comfort=0.3):
    return (w_fare * (1 - cab.fare / 600)
            + w_eta * (1 - cab.eta / 20)
            + w_comfort * cab.comfort / 10)

def choose_cab(cabs, passengers):
    suitable = [cab for cab in cabs if cab.seats >= passengers]
    return max(suitable, key=utility) if suitable else None

cabs = [
    Cab("Micro", 100, 5, 5, 4),
    Cab("Mini", 200, 4, 6, 4),
    Cab("Sedan", 300, 7, 8, 4),
    Cab("Prime", 400, 9, 9, 4),
    Cab("Share", 110, 12, 3, 2),
]

for cab in cabs:
    print(f"{cab.kind:6} utility = {utility(cab):.3f}")
best = choose_cab(cabs, passengers=2)
print(f"Recommended cab: {best.kind if best else 'None available'}")
```

Python IDLE editor



```
>>>
Micro utility = 0.650
Mini utility = 0.657
Sedan utility = 0.603
Prime utility = 0.530
Share utility = 0.578
Recommended cab: Mini
```

Python IDLE Shell output

E. Interpretation of results

Mini is recommended because its shorter waiting time and higher comfort compensate for a modestly higher fare than Micro. The result can change when the rider's weights or live estimates change. If the rider directly chooses a cab type, that explicit choice should take precedence over the system's ranking.

Question 5 — Least-Cost Delivery Route with UCS

A. Problem understanding

A delivery must travel from warehouse S to warehouse G through the directed network. Each edge has a generalized cost that may represent fuel, time or a weighted combination of both. The task is to find the route with the smallest total edge cost, not simply the fewest edges.

The network edges are: S→A costs 1 and S→G costs 12; A→B costs 3 and A→C costs 1; B→D costs 3; C→D costs 1 and C→G costs 2; D→G costs 3.

Delivery network (red = optimal path $S \rightarrow A \rightarrow C \rightarrow G$, cost 4)

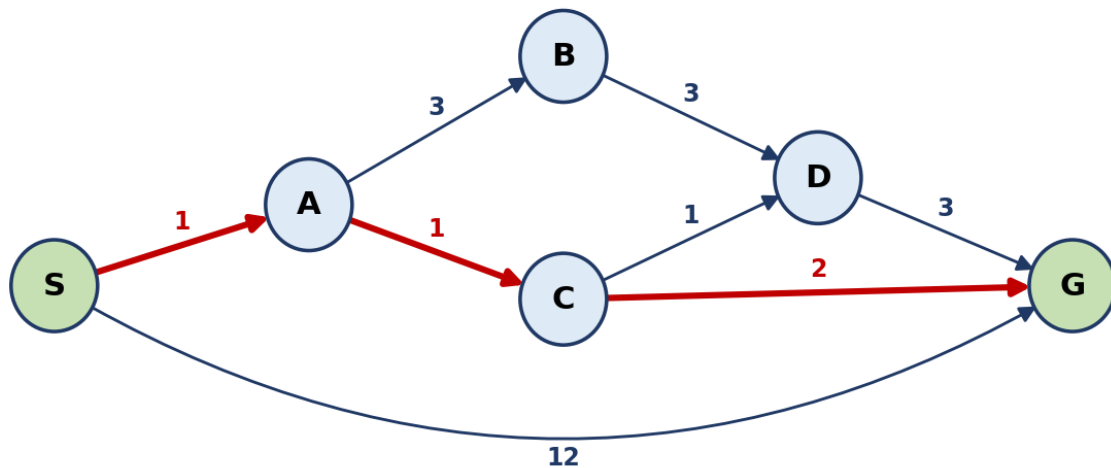


Figure 5.1. Delivery network; the red route $S \rightarrow A \rightarrow C \rightarrow G$ has cost 4.

B. Method selection

Uniform-cost search expands the frontier node with the lowest accumulated cost $g(n)$. All edge costs here are positive, so UCS is complete and optimal. BFS would prefer the one-edge route $S \rightarrow G$ even though its cost is 12, while DFS depends on traversal order and does not guarantee the least-cost route. The goal test must be applied when G is removed from the priority queue for expansion, rather than when it is first generated.

C. Solution process

Starting with S at cost 0, the frontier changes as follows:

1. Expand S ($g = 0$) – add $A(1)$ and $G(12)$.
2. Expand A ($g = 1$) – add $C(2)$ and $B(4)$; the frontier is $C(2)$, $B(4)$, $G(12)$.
3. Expand C ($g = 2$) – add $D(3)$ and a better $G(4)$, replacing the direct $G(12)$ entry.
4. Expand D ($g = 3$) – the route through D gives $G(6)$, so it is worse than the existing $G(4)$.
5. Expand B ($g = 4$) – the route through B gives $D(7)$, worse than the already known $D(3)$.
6. Remove G ($g = 4$) – the goal is now the lowest-cost frontier node, so UCS stops.

UCS Search Tree (#n = expansion order, orange = never expanded / pruned)

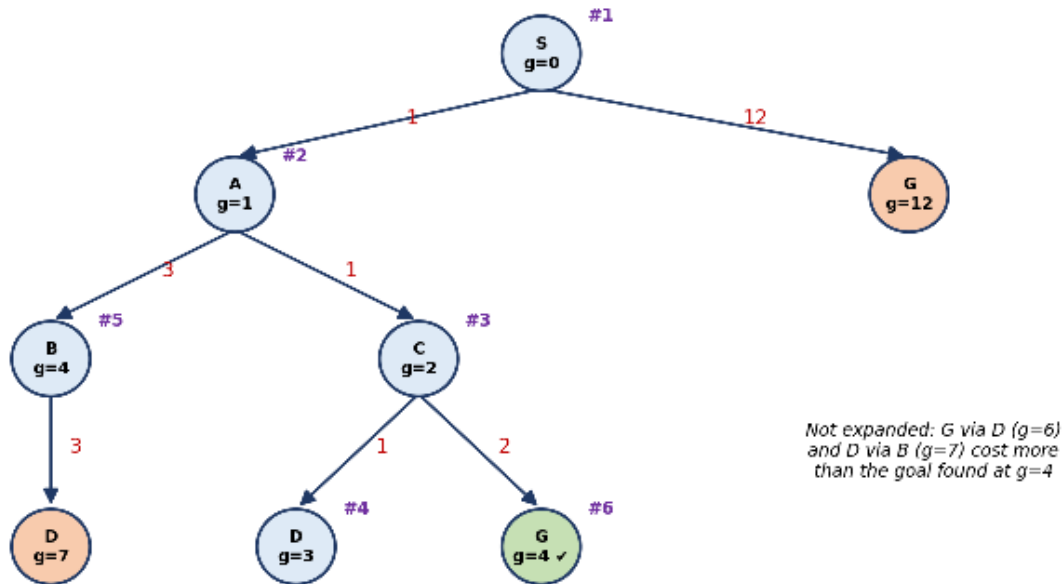


Figure 5.2. UCS expansion tree; the numbers show expansion order and accumulated cost.

The expansion order is S, A, C, D, B, G. B and G both have frontier cost 4; the implementation breaks this tie alphabetically, so B is expanded first. That tie does not affect the optimal route.

D. Accuracy of calculation

The candidate routes check out edge by edge: $S \rightarrow A \rightarrow C \rightarrow G$ costs $1 + 1 + 2 = 4$; $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$ costs $1 + 1 + 1 + 3 = 6$; $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$ costs $1 + 3 + 3 + 3 = 10$; and the direct route $S \rightarrow G$ costs 12. The minimum is 4, matching UCS output.

For this graph, UCS expands six nodes and the maximum frontier contains four entries. With minimum edge cost $\epsilon = 1$ and optimal cost $C^* = 4$, the usual positive-cost search bound is $O(b^{1 + \text{floor}(C^*/\epsilon)})$ in time and space; for this small fixed graph, the observed counts are more informative than the general bound.

The IDLE run expands nodes in the same order and prints $S \rightarrow A \rightarrow C \rightarrow G$ with total cost 4.

Uniform-cost search check: program and run in Python IDLE

The editor shows ucs.py; the Shell screenshot below is the result from running that file in IDLE (Python 3.12.3).

```
ucs.py - /home/subhansh/work/answer-network/ucs.py (3.12.3)
File Edit Format Run Options Window Help

import heapq

graph = {
    'S': {'A': 1, 'G': 12},
    'A': {'G': 3, 'C': 1},
    'B': {'D': 3},
    'C': {'G': 1, 'G': 2},
    'D': {'G': 3},
    'G': {},
}

def uniform_cost_search(start, goal):
    frontier = [(0, start, [start])]
    explored = set()
    order = []
    while frontier:
        cost, node, path = heapq.heappop(frontier)
        if node in explored:
            continue
        explored.add(node)
        order.append(node)
        print(f"Expand {node}: g = {cost}")
        if node == goal:
            return cost, path, order
        for neighbour, edge_cost in graph[node].items():
            if neighbour not in explored:
                heapq.heappush(
                    frontier, (cost + edge_cost, neighbour, path + [neighbour])
                )
    return None, [], order

cost, path, order = uniform_cost_search('S', 'G')
print(f"Optimal path: " + " -> ".join(path))
print(f"Total cost: {cost}")
print(f"Expansion order: {order}")
```

Python IDLE editor

```
Expand S: g = 0
Expand A: g = 1
Expand C: g = 2
Expand D: g = 3
Expand B: g = 4
Expand G: g = 4
Optimal path: S -> A -> C -> G
Total cost: 4
Expansion order: ['S', 'A', 'C', 'D', 'B', 'G']
>>>
```

Python IDLE Shell output

E. Interpretation of results

The least-cost route is $S \rightarrow A \rightarrow C \rightarrow G$, with total cost 4 generalized units. Compared with the direct route, this saves $12 - 4 = 8$ units, or $8/12 \times 100\% = 66.7\%$. This example shows why routing by edge count alone can be expensive. In a real logistics model, the generalized cost could be converted into rupees, minutes, fuel use or a weighted combination.